

● SESIÓN 10 MapStruct: mapping limpio Entity ↔ DTO

"Mapear no es lógica de negocio."

■ Bloque 1 — Problema real

Ahora mismo estás haciendo mapping manual por todas partes:

```
private BookResponseDTO toDTO(Book b) {  
    return new BookResponseDTO(  
        b.getId(),  
        b.getTitle(),  
        b.getAuthor(),  
        b.getCategory().getName()  
    );  
}
```

Y además:

- en findAll()
- en search()
- en create()
- en update()

👉 Eso genera:

Repetición

Código duplicado en múltiples lugares

Errores por cambios

Difícil mantener sincronizado

Código ruidoso en Service

Mezcla infraestructura con negocio



Mensaje clave:

"El mapping es infraestructura, no negocio."



Bloque 2 — Qué es MapStruct

MapStruct genera código Java en compilación (no reflexión), creando un mapper tipo:

- BookMapperImpl (autogenerado)

Ventajas:

- Muy rápido (código generado)
- Sin reflexión
- Centraliza reglas de mapping
- Menos errores

¿Por qué es mejor?

El código se genera en tiempo de compilación, lo que significa cero overhead en runtime y detección temprana de errores de mapping.



Bloque 3 — Dependencias (Maven)



pom.xml

```
<java.version>17</java.version>
<mapstruct.version>1.5.5.Final</mapstruct.version>

<dependency>
  <groupId>org.mapstruct</groupId>
  <artifactId>mapstruct</artifactId>
  <version>${mapstruct.version}</version>
</dependency>

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <configuration>
    <source>${java.version}</source>
    <target>${java.version}</target>
    <annotationProcessorPaths>
      <path>
        <groupId>org.mapstruct</groupId>
        <artifactId>mapstruct-processor</artifactId>
        <version>${mapstruct.version}</version>
      </path>
    </annotationProcessorPaths>
  </configuration>
</plugin>
```

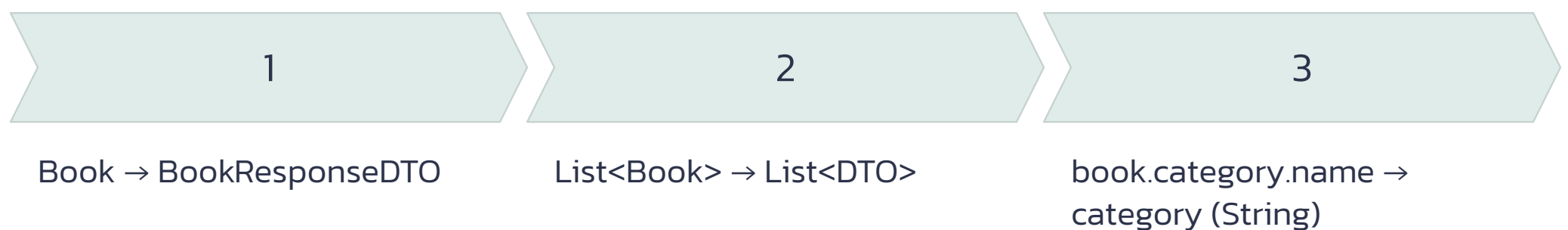
Si usas Lombok, luego te digo el orden correcto de processors (Lombok + MapStruct).

Bloque 4 — Estructura del proyecto

```
com.cladsb1.books
├── controller
│   └── BookController
├── service
│   └── BookService
└── mapper
```

Bloque 5 — Crear BookMapper

Queremos mapear:



 mapper/BookMapper.java

```
package com.cladsb1.books.mapper;

import com.cladsb1.books.dto.response.BookResponseDTO;
import com.cladsb1.books.entity.Book;
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import java.util.List;

@Mapper(componentModel = "spring")
public interface BookMapper {

    @Mapping(target = "category", source = "category.name")
    BookResponseDTO toDTO(Book book);

    List<BookResponseDTO> toDTOList(List<Book> books);
}
```

  `componentModel="spring"` hace que puedas inyectarlo como un bean.



Bloque 6 — Refactor del Service

Antes (manual)

```
return page.getContent()
    .stream()
    .map(this::toDTO)
    .toList();
```

Ahora (MapStruct)

- mapper.toDTO(book)
- o mapper.toDTOList(list)



A) Versión con streams/lambda (para Page content)



service/BookService.java (fragmento)

```
public PageResponseDTO<BookResponseDTO> search(
    BookFilterDTO filter,
    Pageable pageable
){
    Specification<Book> spec = buildSpec(filter);
    Page<Book> page = bookRepository.findAll(spec, pageable);

    return new PageResponseDTO<>(
        page.getContent()
            .stream()
            .map(bookMapper::toDTO)
            .toList(),
        page.getNumber(),
        page.getSize(),
        page.getTotalElements(),
        page.getTotalPages(),
        page.isLast()
    );
}
```



B) Versión sin streams (for clásico)

```
List<BookResponseDTO> content = new ArrayList<>();

for (Book book : page.getContent()) {
    content.add(bookMapper.toDTO(book));
}
```



Bloque 7 — BookService completo (ya con mapper)

Ejemplo compacto (centrado en idea, sin repetir todo el proyecto):

```
@Service
public class BookService {

    private final BookRepository bookRepository;
    private final BookMapper bookMapper;

    public BookService(
        BookRepository bookRepository,
        BookMapper bookMapper
    ){
        this.bookRepository = bookRepository;
        this.bookMapper = bookMapper;
    }

    public PageResponseDTO<BookResponseDTO> search(
        BookFilterDTO filter,
        Pageable pageable
    ){
        Specification<Book> spec = buildSpec(filter);
        Page<Book> page = bookRepository.findAll(spec, pageable);

        return new PageResponseDTO<>(
            page.getContent()
                .stream()
                .map(bookMapper::toDTO)
                .toList(),
            page.getNumber(),
            page.getSize(),
            page.getTotalElements(),
            page.getTotalPages(),
            page.isLast()
        );
    }

    private Specification<Book> buildSpec(BookFilterDTO filter) {
        Specification<Book> spec = Specification.where(null);

        if (filter.title() != null && !filter.title().isBlank()) {
            spec = spec.and(
                BookSpecifications.titleContainsIgnoreCase(filter.title())
            );
        }

        if (filter.author() != null && !filter.author().isBlank()) {
            spec = spec.and(
                BookSpecifications.authorEqualsIgnoreCase(filter.author())
            );
        }

        if (filter.category() != null && !filter.category().isBlank()) {
            spec = spec.and(
                BookSpecifications.categoryNameEqualsIgnoreCase(
                    filter.category()
                )
            );
        }

        return spec;
    }
}
```



Bloque 8 — actualización (@MappingTarget)

MapStruct puede ayudarte a aplicar cambios sobre una entity existente (ideal para PUT).

En el mapper:

```
import org.mapstruct.MappingTarget;

@Mapper(componentModel = "spring")
public interface BookMapper {

    @Mapping(target = "category", ignore = true) // la resolvemos aparte
    void updateEntityFromRequest(
        BookRequestDTO dto,
        @MappingTarget Book entity
    );

    @Mapping(target = "category", source = "category.name")
    BookResponseDTO toDTO(Book book);
}
```

En el service (update):

```
@Transactional
public Optional<BookResponseDTO> update(Long id, BookRequestDTO dto) {
    return bookRepository.findById(id).map(existing -> {
        // 1) MapStruct actualiza campos simples
        bookMapper.updateEntityFromRequest(dto, existing);

        // 2) La relación se resuelve con repository (negocio)
        Category category = resolveCategory(dto.category());
        existing.setCategory(category);

        // 3) Guardamos
        Book saved = bookRepository.save(existing);
        return bookMapper.toDTO(saved);
    });
}
```



Mensaje clave:

"MapStruct rellena campos. El negocio resuelve relaciones."

Bloque 9 — Errores típicos

✗ "No se genera BookMapperImpl"
→ falta annotation processor

✗ IDE no detecta generación
→ activar *annotation processing*

✗ Mapear relaciones sin especificar source
→ (category.name)

✗ Usar MapStruct para lógica de negocio
→ (NO)

Bloque 10 — Cierre

"Mapear no es lógica de negocio."

"El service debe leer como un caso de uso."